25 de Abril de 2025

Granada

ALGORITMOS GREEDY
PRÁCTICA 3
ALGORÍTMICA

Francisco Javier Ortiz Molinero

Alejandro Pérez Pérez

Juan Reche Aguilar

Introducción..... 3

1. Buque mercante..... 4

1.1. Diseño de componentes..... 4

1.2. Diseño del algoritmo..... 4

1.3. Estudio de optimalidad..... 4

1.4. Ejemplo de funcionamiento..... 5

2. Pixel mountain..... 6

2.1. Diseño de componentes..... 6

2.2. Diseño del algoritmo..... 6

2.3. Estudio de optimalidad..... 6

2.4. Ejemplo de funcionamiento..... 6

3. Camino de Euler..... 7

3.1. Diseño de componentes..... 7

3.2. Diseño del algoritmo..... 7

3.3. Estudio de optimalidad..... 8

3.4. Ejemplo de funcionamiento..... 9

Introducción

Los algoritmos greedy, también conocidos como algoritmos voraces, constituyen una de las estrategias más utilizadas en el diseño de algoritmos debido a su simplicidad y eficiencia. Se basan en la idea de tomar decisiones que parecen óptimas en el momento, sin reconsiderar decisiones previas, con la esperanza de que esta elección local conduzca a una solución global óptima. Esta metodología resulta especialmente útil en problemas de optimización, donde se busca maximizar o minimizar alguna función objetivo bajo ciertas restricciones.

La estructura general de un algoritmo greedy incluye cinco componentes fundamentales: una lista de candidatos, un conjunto de candidatos seleccionados, un criterio de selección que define qué elemento tomar a continuación, un criterio de factibilidad que asegura que la solución parcial se mantenga válida, y una función de solución que determina cuándo se ha alcanzado una solución completa. Estos elementos permiten construir un algoritmo que avanza paso a paso, siempre eligiendo la opción más prometedora en cada iteración.

Sin embargo, no todos los problemas pueden resolverse correctamente con un enfoque greedy. Es necesario un análisis previo de optimalidad para asegurar que las decisiones locales conducen efectivamente a una solución óptima. En este trabajo se presentan y analizan distintos algoritmos basados en esta estrategia, aplicados a diferentes contextos clásicos de la teoría de grafos y otros dominios algorítmicos. En cada caso, se incluye el diseño completo del algoritmo, un estudio de su factibilidad y optimalidad, y un ejemplo ilustrativo que muestra su funcionamiento paso a paso.

1. Buque mercante

1.1. Diseño de componentes

Lista de candidatos: todos los contenedores con sus respectivos pesos.

Lista de candidatos usados: contenedores seleccionados hasta el momento que no sobrepasan la capacidad del buque.

Criterio de selección: seleccionar el contenedor de mayor peso entre los no utilizados.

Criterio de factibilidad: añadir un contenedor solo si no excede la capacidad máxima del buque al incluirlo.

Función solución: determina que el conjunto de contenedores en el buque no exceda la capacidad.

Función objetivo: maximizar la suma de los pesos de los contenedores sin excederse de la capacidad máxima del buque.

1.2. Diseño del algoritmo

```
// Función greedy para seleccionar contenedores
pair<vector<int>, int> cargarContenedores(vector<int>& pesos, int capacidad) {
    // Ordenar los pesos de mayor a menor
    sort(pesos.begin(), pesos.end(), greater<int>());

    vector<int> seleccionados;
    int cargaTotal = 0;

    for (int peso : pesos) {
        if (cargaTotal + peso <= capacidad) {
            seleccionados.push_back(peso);
            cargaTotal += peso;
        }
    }

    return {seleccionados, cargaTotal};
}
```

1.3. Estudio de optimalidad

Veamos que el algoritmo Greedy no es óptimo en general con un contraejemplo.

Supongamos los contenedores con los siguientes pesos { 9, 6, 4, 3 } medidos en toneladas y que la capacidad máxima del buque es de 10 toneladas.

Por el algoritmo Greedy se elegirá el de 9 toneladas y ninguno más cabría en el buque porque excedería la capacidad, sin embargo, a simple vista vemos que la solución óptima es elegir el de 6 y el de 4 toneladas.

1.4. Ejemplo de funcionamiento

Veamos cómo funciona el algoritmo Greedy empleado.

Supongamos los contenedores con los siguientes pesos en toneladas: { 7, 2, 10, 3, 5 } y que el buque tiene una capacidad de 15 toneladas.

Lo primero que hace el algoritmo es ordenar el vector de contenedores por peso, es decir, {10, 7, 5, 3, 2}.

Luego se elige el contenedor de mayor peso, en este caso el de 10.

Se comprueba que si añadimos el contenedor al barco no excedemos la capacidad, 10 <= 15. Por tanto añadimos el contenedor de 10 toneladas al buque.

A continuación elegimos el de mayor peso de los no utilizados, el de 7, y comprobamos. 10 + 7 > 15. Luego el contenedor de 7 toneladas no se puede añadir al buque.

Comprobamos ahora con el 5. 10 + 5 = 15, por lo tanto añadimos el contenedor de 5 toneladas al buque.

Como ya tenemos la capacidad máxima del buque ocupada no se pueden añadir más contenedores.

Por tanto la solución por el algoritmo Greedy es { 10, 5 }.

2. Pixel mountain

2.1. Diseño de componentes

Lista de candidatos: las casillas que estén directamente hacia arriba, o hacia arriba-izquierda o arriba-derecha de la casilla en la que nos encontramos en ese momento.

Lista de candidatos usados: cada casilla por la que se decide pasar.

Criterio de selección: seleccionar la casilla de menor riesgo.

Criterio de factibilidad: la casilla esta dentro de la matriz y cumple con los movimientos permitidos.

Función solución: determina que se haya llegado a la cima.

Función objetivo: minimizar la suma de los riesgos de cada casilla.

2.2. Diseño del algoritmo

```cpp
std::vector<int> pixelMountain(const std::vector<std::vector<int>>& mountain) {
    std::vector<int> path;
    int n = mountain.size();
    if (n == 0) return path;
    int m = mountain[0].size();

    double center = (m - 1) / 2.0;

    int current_col = 0;
    int min_cost = mountain[n - 1][0];
    for (int j = 1; j < m; j++) {
        if (mountain[n - 1][j] < min_cost ||
            (mountain[n - 1][j] == min_cost && std::abs(j - center) < std::abs(current_col - center))) {
            min_cost = mountain[n - 1][j];
            current_col = j;
        }
    }
    path.push_back(current_col);

    for (int i = n - 2; i >= 0; i--) {
        int best_col  = current_col;
        int best_cost = mountain[i][current_col];

        for (int dj = -1; dj <= 1; dj++) {
            int next_col = current_col + dj;
            if (next_col >= 0 && next_col < m) {
                int cost = mountain[i][next_col];
                if (cost < best_cost ||
                    (cost == best_cost && std::abs(next_col - center) < std::abs(best_col - center))) {
                    best_cost = cost;
                    best_col  = next_col;
                }
            }
        }
        current_col = best_col;
        path.push_back(current_col);
    }

    return path;
}
```

La idea del algoritmo comienza con elegir primero la casilla de la base en la que queremos comenzar la subida y elegimos la que tenga menor riesgo. A partir de aquí, usamos un ciclo for que iterará sobre el resto de la matriz. La idea esta en saber en que columna estamos y comprobar que numero dentro de los movimientos permitidos es el que más nos combiene, es por eso que también miramos cual esta más cerca del centro para que despues tengamos más candidatos.

2.3. Estudio de optimalidad

Elalgoritmo greedy implementado en este código no siempre consigue el mejor resultado. Esto lo podemos demostrar con el ejemplo mostrado en el enunciado del ejercicio en el guión de practicas:

2	8	9	**5**	8
4	4	6	**2**	3
5	7	5	6	**1**
3	2	5	**4**	8

Como podemos ver en negrita, esta marcado el mejor camino hasta la cima. Sin embargo, el algoritmo greedy seguiria el siguiente camino {2, 5, 2, 5}.

2.4. Ejemplo de funcionamiento

Podemos ver detenidamente el funcionamiento del algoritmo con la matriz anterior.

Instancia 0:

- Lista de candidatos = {3, 2, 5, 4, 8}
- Por el criterio de selección elegimos {2}
- Lista de candidatos usados = {2}

Instancia 1:

- Lista de candidatos = {5, 7, 5}
- Por el criterio de selección elegimos {5 (el de la derecha por ser más cercano al centro)}
- Lista de candidatos usados = {2, 5}

Instancia 2:

- Lista de candidatos = {4, 6, 2}
- Por el criterio de selección elegimos {2}
- Lista de candidatos usados = {2, 5, 2}

Instancia 3:

- Lista de candidatos = {9, 5, 8}
- Por el criterio de selección elegimos {5}
- Lista de candidatos usados = {2, 5, 2, 5}

En este momento, por la funcion solución se detiene la función y hemos obtenido la solución final que es igual a la lista de candidatos usados.

3. Camino de Euler

3.1. Diseño de componentes

Lista de candidatos: Las aristas disponibles que aún no han sido recorridas.

Lista de candidatos usados: Las aristas que ya han sido incluidas en el camino de Euler.

Criterio de selección: Elegir la próxima arista desde el vértice actual que no haya sido usada y cuya eliminación no desconecte el grafo (salvo que sea la única opción).

Criterio de factibilidad: Asegurar que al seleccionar una arista, se sigue teniendo un camino válido y el grafo restante sigue siendo conexo (excepto al final).

Función solución: Un recorrido que comienza y termina en el mismo vértice y contiene todas las aristas exactamente una vez.

Función objetivo: Completar el circuito de Euler sin repetir aristas y usando todas.

3.2. Diseño del algoritmo

Este es el planteamiento del algoritmo:

1. Elegir un vértice de inicio arbitrario *v*.
2. Crear una pila *stack* y una lista *circuit* (vacía).
3. Mientras la pila no esté vacía o haya aristas salientes desde *v*:
    a. Si *v* tiene aristas no usadas:
        i. Agregar *v* a *stack*.
        ii. Escoger una arista (v, u) que no haya sido usada.
        iii. Marcar (v, u) como usada.
        iv. Moverse a *u* (v = u).
    b. Si *v* no tiene aristas no usadas:
        i. Agregar *v* a *circuit*.
        ii. Sacar el último vértice de *stack* y asignarlo a *v*.
4. Invertir *circuit* y devolverlo.

Y esta el la implementación del mismo:

```cpp
class Grafo {
private:
    int V; // Número de vértices
    unordered_map<int, list<int>> adyacente; // Mapa de adyacencias

public:
    Grafo(int V) : V(V) {}

    void agregarArista(int u, int v) {
        adyacente[u].push_back(v);
        adyacente[v].push_back(u); // No dirigido
    }

    void removerArista(int u, int v) {
        adyacente[u].remove(v);
        adyacente[v].remove(u);
    }

    vector<int> circuitoEuler(int inicio) {
        stack<int> pila;
        vector<int> circuito;
        int actual = inicio;

        while (!pila.empty() || !adyacente[actual].empty()) {
            if (adyacente[actual].empty()) {
                circuito.push_back(actual);
                actual = pila.top(); pila.pop();
            } else {
                pila.push(actual);
                int siguiente = adyacente[actual].front();
                removerArista(actual, siguiente);
                actual = siguiente;
            }
        }

        circuito.push_back(actual); // Agregar el último
        return circuito;
    }
};
```

3.3. Estudio de optimalidad

Teorema (Hierholzer, 1873):

Si un grafo no dirigido es conexo y todos sus vértices tienen grado par, entonces tiene un circuito de Euler. Y el algoritmo de Hierholzer siempre encuentra uno.

Prueba intuitiva:

1. Partimos de cualquier vértice y recorremos aristas no utilizadas hasta volver al vértice de inicio.
2. Si aún hay aristas sin usar, significa que hay un vértice en el ciclo actual con aristas sin usar.
3. Desde ahí, construimos un nuevo ciclo y lo insertamos en el original.

4. Como todos los vértices tienen grado par, siempre es posible salir por otra arista sin dejar vértices colgados.

Este proceso se repite hasta usar todas las aristas y se garantiza que se forma un ciclo cerrado.

En conclusión, el algoritmo es óptimo porque encuentra un circuito de Euler en cualquier grafo que lo tenga, y no omite ninguna arista. No existe un circuito mejor ni más corto porque todos los circuitos de Euler recorren exactamente todas las aristas una vez.

Tenemos que el algoritmo puede fallar, pero solo si el grafo no es de Euler (por ejemplo, tiene vértices de grado impar o es disconexo). Pero esto no es culpa del algoritmo, sino de que no existe solución.

3.4. Ejemplo de funcionamiento

Vértices: {1, 2, 3}

Aristas: {(1,2), (2,3), (3,1)}

Grados: 1:2, 2:2, 3:2

Tenemos que es un grafo de Euler, ya que todos los nodos del grafo tienen nodo par, y además, es conexo.

Vemos el funcionamiento del algoritmo paso a paso:

1. v = 1
2. stack = [], circuit = []
3. (1,2) → stack = [1], v = 2
4. (2,3) → stack = [1,2], v = 3
5. (3,1) → stack = [1,2,3], v = 1 (ya no quedan más)
6. Agregar 1 a circuit → circuit = [1], stack = [1,2]
7. Agregar 3 a circuit → circuit = [1,3], stack = [1]
8. Agregar 2 a circuit → circuit = [1,3,2], stack = []
9. Agregar 1 a circuit → circuit = [1,3,2,1]

Circuito de Euler: 1 → 2 → 3 → 1